Great 👍 If you want a **mid-level portfolio project finished in 15 days**, the key is **structured execution**. Each day should produce something visible in the project.

Below is a **realistic 15-day plan** for your **CareFlow – Smart NHS Appointment & Queue Optimisation Platform** using:

- **React + TypeScript**
- **Node.js + Express**
- **MySQL**
- **JWT + bcrypt**
- **Docker**
- **Azure deployment**

Try to spend **4–6 focused hours daily**.

📅 15-Day Development Plan

🟢 Day 1 — Project Setup & Architecture

Goal: Prepare full project structure.

Tasks:

- Create GitHub repository
- Setup project folders

```
careflow/
  frontend/
  backend/
  database/
```

Backend:

src/

controllers/

routes/

middleware/

services/

models/

config/

Frontend:

src/

components/

pages/

services/

hooks/

types/

Install core dependencies.

Backend:

express

mysql2

jsonwebtoken

bcrypt

cors

dotenv

Frontend:

react

typescript

axios

react-router-dom

tailwind

🟢 Day 2 — Database Design

Create MySQL database.

Tables:

- Clinics
- Users
- Doctors
- Patients
- Appointments
- Queue
- Symptoms

Write SQL schema.

Example:

```
CREATE TABLE clinics (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(255),
 latitude DECIMAL(9,6),
 longitude DECIMAL(9,6)
);
```

Test connection from Node backend.

🟢 Day 3 — Backend Setup

Setup Express server.

Create:

server.ts

Add:

- database connection
- middleware
- error handling

Test API:

GET /health

Return:

API running

🟢 **Day 4 — Authentication System**

Implement:

- User registration
- Login
- bcrypt password hashing
- JWT token creation

Endpoints:

POST /auth/register

POST /auth/login

Test using Postman.

🟢 **Day 5 — JWT Middleware**

Create authentication middleware.

Verify token.

Authorization: Bearer TOKEN

Attach user data to request.

Add role middleware:

admin

doctor

patient

🟢 **Day 6 — Clinics & Doctors API**

Create endpoints:

GET /clinics

POST /clinics

GET /doctors

Admin can create doctors.

Store doctor schedules.

🟢 **Day 7 — Patient Management**

Create patient APIs.

POST /patients

GET /patients

GET /patients/:id

Patients belong to clinic.

Doctors can view assigned patients.

🟢 **Day 8 — Appointment System**

Create appointment endpoints.

POST /appointments

GET /appointments

PATCH /appointments/:id

Features:

- create appointment
- cancel appointment
- check availability

🟢 **Day 9 — Smart Triage Logic**

Create symptom submission API.

POST /triage

Example scoring:

breathing difficulty → +5

chest pain → +5

fever → +2

Calculate priority:

score ≥ 8 → emergency

score ≥ 4 → urgent

else → normal

Save to database.

🟢 **Day 10 — Nearest Appointment Algorithm**

Implement logic:

1. Get patient location
2. Find nearby clinics
3. Query available slots
4. Return earliest results

Endpoint:

GET /appointments/nearest

Return:

Clinic A – 4:30pm

Clinic B – 5:00pm

🟢 **Day 11 — Emergency Queue System**

Create queue system.

Table:

queue

Logic:

Patients added to queue with priority.

Doctor dashboard:

GET /queue

Sorted by:

priority + arrival_time

🟢 **Day 12 — Frontend Setup**

Create React TypeScript app.

Setup:

- routing
- axios API client
- auth context
- protected routes

Pages:

Login

Register

Dashboard

🟢 **Day 13 — Main UI Features**

Build pages:

Patient:

- symptom form
- appointment booking
- queue status

Doctor:

- patient list
- appointment dashboard

Admin:

- clinic management

🟢 **Day 14 — Deployment & Docker**

Create Dockerfile.

Deploy backend:

Azure App Service

Deploy frontend:

Azure Static Web Apps or Vercel

Database:

Azure MySQL

Test production API.

🟢 **Day 15 — Polish & Portfolio**

Add:

- README

- screenshots
- architecture diagram
- API documentation

README sections:

- project overview
- features
- tech stack
- architecture
- setup instructions

📊 Final Portfolio Features

Your project will include:

✓ React TypeScript frontend
✓ Node.js backend
✓ MySQL relational database
✓ JWT authentication
✓ bcrypt password security
✓ Smart triage logic
✓ Nearest appointment search
✓ Emergency queue management
✓ Docker containerisation
✓ Azure cloud deployment

That is **mid-level portfolio quality**.

💡 One Last Tip

Do **NOT try to make it perfect UI**.

Recruiters care more about:

- architecture

- backend logic
- database design
- deployment

✅ If you want, I can also show you next:

- **Complete database schema (production-ready)**
- **System architecture diagram**
- **Frontend page wireframes**
- **How to explain this project in interviews**

Those will make your project look **much stronger to UK recruiters**.